

Deep Learning 1 - VAE's and Adversarial Attacks

Matúš Halák - 13626574 (UvA Student ID)

December 2025

1 Variational Autoencoders

1.1 The Generative Decoder

Question 1.1: Steps needed to sample and image given a decoder f_θ Given a decoder f_θ , to sample an image we first need an input for the decoder. Even without having an encoder, we can simply sample the latent $z \in \mathbb{R}^D$ from a standard normal distribution $z \sim \mathcal{N}(0, I_D)$ and pass this as input into the decoder.

The decoder will then produce for each of the M image pixels a probability distribution over the K possible categorical values for each pixel.

$$f_\theta(z) \in \mathbb{R}^{M \times K} \quad \text{s.t.} \quad \sum_{k=1}^K [f_\theta(z)]_{mk} = 1 \quad \forall m \in (1, \dots, M)$$

Finally, the sampled pixel value $\hat{x}^{(m)}$ for each of the M pixels is drawn from the categorical distribution over all possible categorical values for each pixel (0 to $K - 1$), with probabilities given by the decoder.

$$\hat{x}^{(m)} \sim \text{Cat}(x^{(m)} | f_\theta(z)_{m,:}) \in \{0, 1, \dots, K - 1\}$$

Question 1.2: Inefficiency and poor scaling of Monte-Carlo Integration The reason why Monte-Carlo Integration with samples from $p(z_n)$ is highly inefficient for approximating $\log p(x_n)$, is that $p(z_n)$ is an uninformed prior and most samples drawn from it fall far away from the small region where the data-informed posterior $p(z_n | x_n)$ has high density - which is the region where the data likelihood $p(x_n | z_n)$ is large. Therefore, the data likelihood $p(x_n | z_n)$ for most samples drawn from $p(z_n)$ is tiny and contributes very little to the integral, making the procedure highly sample-inefficient. In higher dimensions, the data-informed posterior region $p(z_n | x_n)$ covers an exponentially small fraction of the total volume covered by the prior $p(z_n)$. Thus, the probability that a sample randomly drawn from the uninformed prior lands in the posterior region decreases exponentially with growing latent dimensionality, meaning the number of required samples for an accurate Monte-Carlo estimate grows exponentially as well.

1.2 The Encoder

Question 1.3: Why evidence lower bound The equation

$$\log p(x_n) - D_{KL}(q_\phi(z_n | x_n) || p(z_n | x_n)) = \mathbb{E}_{q_\phi(z_n | x_n)}[\log p_\theta(x_n | z_n)] - D_{KL}(q_\phi(z_n | x_n) || p(z_n)) \quad (1)$$

contains ELBO as its right-hand side

$$ELBO = \mathbb{E}_{q_\phi(z_n|x_n)}[\log p_\theta(x_n|z_n)] - D_{KL}(q_\phi(z_n|x_n)||p(z_n))$$

From Equation 1, we see that ELBO is a *lower bound* because Equation 1 has the form $\log p(x_n) - D_{KL} = ELBO$. Because the KL divergence is non-negative $D_{KL}(q||p) \geq 0 \forall q, p$, it must be true that $ELBO \leq \log p(x_n)$, making ELBO a *lower bound* on $\log p(x_n)$. In other words, subtracting a non-negative KL divergence from $\log p(x_n)$ can only decrease its value, i.e. $\log p(x_n) \geq \log p(x_n) - D_{KL}(q||p)$.

Question 1.4: ELBO as variational distribution approaches true posterior

From Equation 1, we also notice that as $q_\phi(z_n|x_n) \rightarrow p(z_n|x_n)$, the ELBO becomes an increasingly better and better approximation of the data log-likelihood (since ELBO only differs from the true log-likelihood by the divergence between the approximate and exact latent posterior). In the limit, when $q_\phi(z_n|x_n) \approx p(z_n|x_n)$ and $D_{KL}(q_\phi(z_n|x_n)||p(z_n|x_n)) \approx 0$, then $\log p(x_n) \approx ELBO$.

1.3 Optimization objective

Question 1.5: Reconstruction loss and regularization loss

$$\mathcal{L}_n^{recon} = -\mathbb{E}_{q_\phi(z_n|x_n)}[\log p_\theta(x_n|z_n)]$$

We call this a **reconstruction loss** because this term forces the decoder f_θ to faithfully **reconstruct** the original input data x_n from latent representations of the input data z_n sampled from the encoder q_ϕ , by penalizing reconstructions that assign low likelihood to ground-truth original input data.

$$\mathcal{L}_n^{reg} = D_{KL}(q_\phi(z_n|x_n)||p(z_n))$$

This term is called the **regularization loss**, because it **regularizes** the encoder's approximate posterior q_ϕ to adhere closely to the prior p , enforcing a structured, continuous organization of the latent space.

Question 1.6: Estimating the regularization term for arbitrary priors Given any prior distribution with a known probability density function $p(z)$, we can estimate the regularization term $\mathcal{L}^{reg}$ with a Monte-Carlo approximation of KL divergence:

$$D_{KL}(q_\phi(z_n|x_n)||p(z_n)) = \mathbb{E}_{z_n \sim q_\phi(z|x)}[\log \frac{q_\phi(z_n|x_n)}{p(z_n)}] \approx \frac{1}{S} \sum_{s=1}^S \log \frac{q_\phi(\hat{z}_n^{(s)}|x_n)}{p(\hat{z}_n^{(s)})} \quad \hat{z}_n^{(s)} \sim q_\phi(z_n|x_n)$$

Since the encoder q_ϕ parametrizes a variational distribution over z (the encoder estimates the parameters of a distribution with a known probability density function), we can sample from the variational distribution S times to obtain latent samples $\hat{z}_n^{(s)}$ and evaluate the probability density of those samples under the variational distribution $q_\phi(\hat{z}_n^{(s)}|x_n)$, as well as, under any prior distribution with a known probability density function $p(\hat{z}_n^{(s)})$. This allows us to compute a Monte-Carlo approximation of $D_{KL}(q_\phi||p)$, giving an estimate for $\mathcal{L}^{reg}$ for any q_ϕ and p distributions with available probability density functions.

1.4 The Reparameterization Trick

Question 1.7: Reparameterization trick and gradients The direct sampling operation $z \sim \mathcal{N}(z|\mu_\phi(x), \Sigma_\phi(x))$ does not explicitly expose the influence of the μ_ϕ and Σ_ϕ terms on z for backpropagation. Instead, μ_ϕ and Σ_ϕ are implicitly involved in the stochastic sampling operation by defining the sampling distribution. Because of the randomness, backpropagation has no way to deterministically attribute errors from z to μ_ϕ and Σ_ϕ .

In contrast, when using the reparameterization trick:

$$\mathbf{z}_n = \mu_\phi(\mathbf{x}_n) + \sigma_\phi(\mathbf{x}_n) \odot \epsilon \in \mathbb{R}^D \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$$

only the sampling of random "noise" ϵ is stochastic, while μ_ϕ and σ_ϕ are fully-exposed terms in the computation graph, easy to backpropagate through. Since we only care about gradients with respect to μ_ϕ and σ_ϕ this fully solves the problem, while producing solutions equivalent to direct sampling. The ϵ sampled in the forward pass is simply stored as a constant in memory and used for backpropagation through σ_ϕ .

1.5 Building a VAE

Question 1.8: MNIST VAE Here we built a CNN-based Variational Autoencoder and trained it on the MNIST dataset for 80 epochs. For the encoder and decoder CNN architectures, we adapted and extended the Tutorial 9 autoencoder template with adding Layer normalization before each nonlinearity, as well as, with replacing Convolution layers that do not change shape of previous layer's outputs by Residual blocks. Figure 1 shows the bits-per-dimension (bpd) loss evolution over the course of training, as well as, the final bpd calculated on the MNIST test set ($\text{bpd}_{\text{test}} = 0.512$).

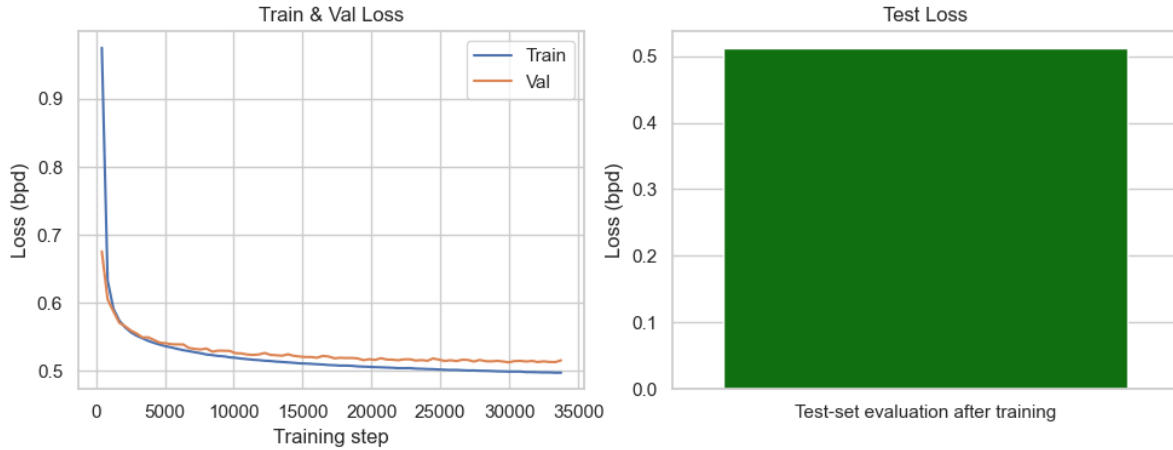


Figure 1: Bits per dimension score (bpd) for training, validation and test MNIST sets.

Question 1.9: Quality of generated samples over course of training In Figure 2 we show random samples generated by our model over the course of training.



(a) Samples before training (b) Samples after 10 epochs (c) Samples after 80 epochs

Figure 2: Model-generated samples across training

Before training, the model starts out by generating pure noise from random samples drawn from a standard normal distribution (Figure 2a). At this stage, the decoder has not learned how to map samples from the latent space to images. Already after 10 epochs, network has learned to organize the latent space, such that most random samples from the standard normal prior are mapped to realistically-looking images of hand-drawn digits - corresponding to sampled locations in the organized latent space (Figure 2b). Further

training up to 80 epochs incrementally improves the clarity of the generated samples (Figure 2c), but the biggest gains in organizing the latent space happen in the early epochs. Nevertheless, even after 80 epochs, some samples include strange noisy artifacts, or are not clearly identifiable as 0-9 digits.

Question 1.10: MNIST Manifold Figure 3 shows the learned MNIST data manifold after training the CNN-VAE with 2-dimensional latent space for 80 epochs.

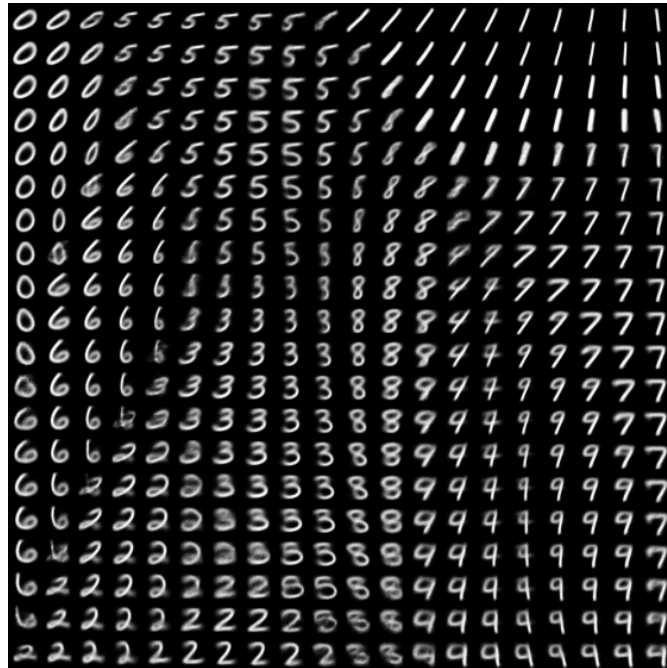


Figure 3: Learned MNIST manifold in 2-dimensional latent space.

The structure of the learned manifold is remarkably similar to Figure 4b of the original VAE paper (Kingma and Welling, 2014), despite using somewhat different architectures. Namely, we notice that clusters of digits 3 and 8 occupy central regions of the distribution with the highest density. This emergent organization of latent space is intuitive, since digits 3 and 8 can smoothly morph into each other and many other digits, ensuring smoothness of the latent space around regions with highest probability mass. In contrast, highly distinct digit shapes such as 0 and 1 cluster occupy borders of the distribution, because transitions between these distinct digits and other digits are less smooth, and placing them more centrally would disturb the smoothness of the latent space.

1.6 β -VAEs

Question 1.11: Effect of increasing β on latent representations As β increases, the relative importance of the regularization loss term $\mathcal{L}^{reg}$ increases. During training, backpropagation will prioritize minimizing the larger $\mathcal{L}^{reg}$ term by minimizing $D_{KL}(q_\phi(z|x)||p(z))$ effectively pushing $q_\phi(z|x)$ closer to $p(z)$. The resulting approximate posterior latent representations produced by $q_\phi(z|x)$ will be more constrained by the uninformed $p(z)$ prior, meaning the latent representations will become less data-dependent. While this will make the latent representation space more structured (closer to the uninformed gaussian prior), this might come at the cost of reconstruction quality.

2 Adversarial Attacks

2.1 Fast Gradient Sign Method (FGSM)

2.1.1 FGSM attack

ResNet18 Model	Config	Test accuracy	Test FGSM accuracy
Pretrained	Default	0.92	0.444
Fine-tune Pretrained	Default + augmentation	0.93	0.451
From scratch	Default	0.68	0.017
From scratch	Default + augmentation	0.68	0.074

Table 1: FGSM attack experiments.

Table 1 shows the results of initial experiments with FGSM attacks ($\epsilon = 0.1$). Note that in all cases test accuracy on the FGSM-perturbed images is significantly lower than on the unperturbed images, both for pretrained and trained-from-scratch networks.

- **Random perturbation vs FGSM perturbation of same size ϵ .** The reason why adding random ϵ -sized perturbation would not have the same effect is that by chance, some of the random perturbations would take a ϵ step up the gradient and some of the random perturbations would take a ϵ step down the gradient. This means the pixel perturbations that would make correct classification easier and the adversarial pixel perturbations would essentially average out, since the random perturbations would not align with the gradient. In contrast, FGSM perturbations take a step up the gradient with respect to the loss for all pixels, making the classification accuracy of the worst-case perturbed image decidedly worse.
- **Why FGSM attack built using gradients from one model works on another model trained on a different subset of the same dataset.** This is likely because both models are trained on the same task and thus have similar gradients, learn similar decision boundaries, as well as, similar low-level features in the early layers. Therefore, attacks exploiting the gradients and learned decision boundaries of model A will likely work on the similar gradients and decision boundaries for model B trained on the same task.
- **Data augmentation.** We note that fine-tuning or training with simple data augmentation (random flipping and cropping) slightly improves test accuracy to adversarial examples (pretrained $44.4 \rightarrow 45.1$ %, from scratch $1.7 \rightarrow 7.4$ %), while maintaining or slightly improving accuracy on unperturbed images. This slight improvement may be attributed to the network learning somewhat more robust features and invariances to flip and crop perturbations of the images, which helps with generalization and adversarial robustness as well. However, since the improvements are minor, only using data augmentation is not a sufficient adversarial defense.

2.1.2 FGSM defense

Table 2 shows the results of experiments with FGSM attacks and defenses ($\alpha = 0.5, \epsilon = 0.1$). FGSM attacks led to noticeably lower perturbed test accuracy in pretrained (44.4%) and from-scratch (1.7%) undefended networks compared to unperturbed test accuracies (92% and 68%). Adversarial training with FGSM adversarial loss led to a drop in unperturbed image test accuracy both when fine-tuning pretrained networks (91%) and training from scratch (60%). However, adversarial training only improved adversarial FGSM accuracy when fine-tuning of pretrained networks was paired with data augmentation (60.4%), or when training networks from scratch. Interestingly, networks trained from scratch with adversarial loss performed better without data augmentation (unperturbed 60%, FGSM 40%) than with data augmentation (unperturbed 53%, FGSM 37.8%).

ResNet18 Model	Config	Test accuracy	Test FGSM accuracy
Pretrained	Default	0.92	0.444
Fine-tune Pretrained	FGSM loss	0.91	0.444
Fine-tune Pretrained	FGSM loss + augmentation	0.91	0.604
From scratch	Default	0.68	0.017
From scratch	FGSM loss	0.60	0.4
From scratch	Default + augmentation	0.68	0.074
From scratch	FGSM loss + augmentation	0.53	0.378

Table 2: FGSM defense experiments.

- **Tradeoff between defending and not defending adversarial attacks.** Adversarial training introduces a trade-off between unperturbed image and adversarial image accuracy. By not fully relying on unperturbed gradients and also instead taking into account worst-case perturbations at each step, we by definition sacrifice some performance on the unperturbed images (since we are not taking full advantage of the unperturbed gradients during backpropagation, but are trying to optimize on a harder task). On the other hand, considering worst-case perturbations makes the networks more robust against adversarial attacks, increasing adversarial performance.

2.2 Projected Gradient Descent (PGD)

Table 3 shows the results of experiments with PGD attacks and defenses ($\alpha = 0.01, \epsilon = 0.1$). PGD attacks led to noticeably lower perturbed test accuracy in pretrained (8.6%) and from-scratch (0%) undefended networks compared to FGSM attacks in undefended pretrained and from-scratch networks (Table 2, 44.4% and 1.7%), showing that PDG is a significantly stronger attack than FGSM. Adversarial training with PGD examples led to a drop in unperturbed image test accuracy and improved PGD attack accuracy both when fine-tuning pretrained networks (85% and 38.7%) and training from scratch (49% and 26.4%). Finally, similar to FGSM (Table 2), supplementing adversarial training with data augmentation improved performance when fine-tuning pretrained networks, but slightly impaired performance when training from scratch.

ResNet18 Model	Config	Test accuracy	Test PGD accuracy
Pretrained	Default	0.92	0.086
Fine-tune Pretrained	PGD defense	0.85	0.387
Fine-tune Pretrained	PGD defense + augmentation	0.87	0.408
From scratch	Default	0.68	0.0
From scratch	PGD defense	0.49	0.264
From scratch	Default + augmentation	0.68	0.04
From scratch	PGD defense + augmentation	0.44	0.252

Table 3: PGD experiments.

- **Adversarial loss vs Adversarial examples for PGD.** An adversarial loss (in style of the FGSM adversarial loss) considers a λ -weighed sum of the loss on the original batch of images X , and the loss on the adversarial counterparts X_{adv} .

$$L_{adv}(X) = \lambda L(X) + (1 - \lambda)L(X_{adv})$$

If $\lambda = 1/2$, then the adversarial loss is simply the average loss calculated over both all the original images in the batch and all their adversarial counterparts. Thus, in case the losses to the original and adversarial images in the batch are weighed equally ($\lambda = 1/2$) using an adversarial loss is equivalent to adding adversarial images to the batch and computing one loss over the new big batch. However, when the two losses are not weighed equally ($\lambda \neq 1/2$), simply adding adversarial examples to the batch will lead to different solutions than using an adversarial loss with an explicit weighing of original and adversarial losses.

- **FGSM vs PGD.** *Advantage PGD over FGSM:* PGD produces a smaller less noticeable perturbation that does more damage to network performance relative to FGSM, since PGD iteratively refines the perturbation while keeping it minimally noticeable (within ϵ -ball of original image). *Advantage FGSM over PGD:* FGSM is faster and more computationally efficient relative to PGD since FGSM is a one-step attack without iterative refinement. This also means that adversarial training with FGSM defense is significantly faster.